

# Data Structures and Algorithms

GÉZA HORVÁTH

Second Lecture

# Lecture topics

- 1 Introductory examples of the design and analysis of algorithms.
- 2 Asymptotic behavior of functions, asymptotic notation, further examples of simple algorithms.
- 3 Basic notions concerning data structures: modeling, abstraction. Set, multiset, array, matrix.
- 4 Elementary data structures: stacks, queues, lists.
- 5 Tables, hash tables, hash functions, collisions.
- 6 Trees, binary trees, binary search trees, traversal, searching, insertion, deletion.
- 7 Balanced binary search trees, AVL trees.
- 8 Red-black trees.
- 9 B-trees.
- 10 Graphs, traversal, shortest path.
- 11 Parallel algorithms: matrix multiplication, merge sort.
- 12 Decision problems, the complexity classes P and NP.
- 13 Summary.

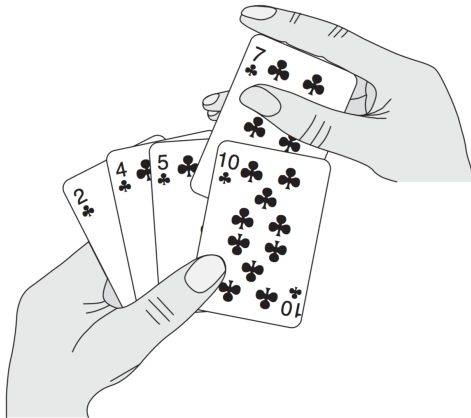
# Sorting problem

**Input:** A sequence of  $n \geq 2$  numbers  $\langle a_1, a_2, \dots, a_n \rangle$ .

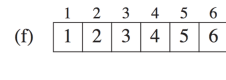
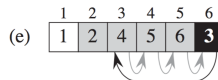
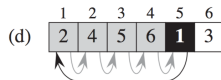
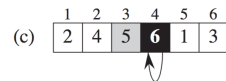
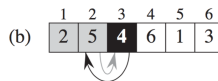
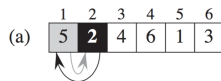
**Output:** A permutation (reordering)  $\langle a'_1, a'_2, \dots, a'_n \rangle$  of the input sequence such that  $a'_1 \leq a'_2 \leq \dots \leq a'_n$ .

**Example:** Given the input sequence  $\langle 31, 41, 59, 26, 41, 58 \rangle$ , a sorting algorithm returns as output the sequence  $\langle 26, 31, 41, 41, 58, 59 \rangle$ . (Book page 5.)

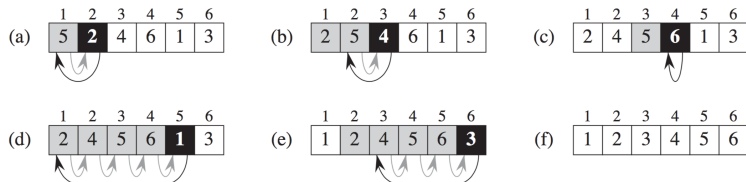
# Insertion sort - basic idea



# Insertion sort - example



# Insertion sort - pseudocode

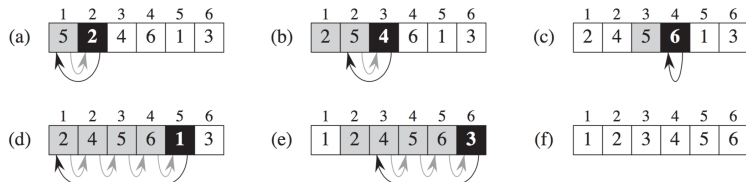


## INSERTION-SORT( $A$ )

```

1  for  $j = 2$  to  $A.length$ 
2       $key = A[j]$ 
3      // Insert  $A[j]$  into the sorted sequence  $A[1..j-1]$ .
4       $i = j - 1$ 
5      while  $i > 0$  and  $A[i] > key$ 
6           $A[i+1] = A[i]$ 
7           $i = i - 1$ 
8       $A[i+1] = key$ 
  
```

# Analysis of insertion sort



## INSERTION-SORT(*A*)

```

1  for  $j = 2$  to  $A.length$ 
2     $key = A[j]$ 
3    // Insert  $A[j]$  into the sorted
      sequence  $A[1..j - 1]$ .
4     $i = j - 1$ 
5    while  $i > 0$  and  $A[i] > key$ 
6       $A[i + 1] = A[i]$ 
7       $i = i - 1$ 
8     $A[i + 1] = key$ 

```

*cost**times* $c_1$  $n$  $c_2$  $n - 1$ 

0

 $n - 1$  $c_4$  $n - 1$  $c_5$  $\sum_{j=2}^n t_j$  $c_6$  $\sum_{j=2}^n (t_j - 1)$  $c_7$  $\sum_{j=2}^n (t_j - 1)$  $c_8$  $n - 1$

# Analysis of insertion sort

INSERTION-SORT( $A$ )

1 **for**  $j = 2$  **to**  $A.length$

2      $key = A[j]$

3     // Insert  $A[j]$  into the sorted  
       sequence  $A[1..j-1]$ .

4      $i = j - 1$

5     **while**  $i > 0$  and  $A[i] > key$

6          $A[i+1] = A[i]$

7          $i = i - 1$

8      $A[i+1] = key$

*cost*

*times*

$c_1$

$n$

$c_2$

$n - 1$

0

$n - 1$

$c_4$

$n - 1$

$c_5$

$\sum_{j=2}^n t_j$

$c_6$

$\sum_{j=2}^n (t_j - 1)$

$c_7$

$\sum_{j=2}^n (t_j - 1)$

$c_8$

$n - 1$

$$\begin{aligned}
 T(n) = & c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) \\
 & + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1) .
 \end{aligned}$$



# Analysis of insertion sort – best case: already sorted

$$\begin{aligned}
 T(n) = & c_1n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) \\
 & + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1) .
 \end{aligned}$$

**Best case:**  $t_j = 1$  for each  $j = 2, \dots, n$ .

$$\begin{aligned}
 T(n) &= c_1n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) \\
 &= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) .
 \end{aligned}$$

$$a = c_1 + c_2 + c_4 + c_5 + c_8$$

$$b = -1 * (c_2 + c_4 + c_5 + c_8)$$

$$T(n) = a * n + b$$

# Analysis of insertion sort – worst case: reverse sorted order

$$\begin{aligned}
 T(n) = & c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) \\
 & + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1) .
 \end{aligned}$$

**Worst case:**  $t_j = j$  for each  $j = 2, \dots, n$ .

$$\sum_{j=2}^n j = \frac{n(n+1)}{2} - 1$$

and

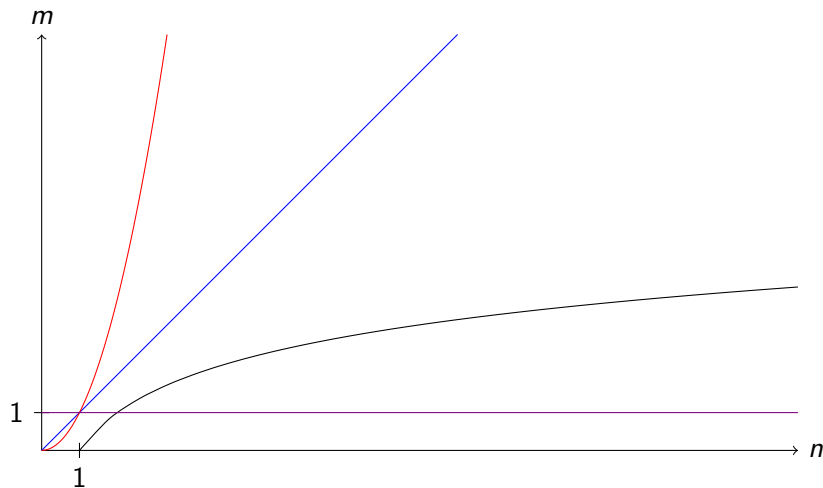
$$\sum_{j=2}^n (j-1) = \frac{n(n-1)}{2}$$

# Analysis of insertion sort – worst case: reverse sorted order

$$\begin{aligned}
 T(n) &= c_1n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right) \\
 &\quad + c_6\left(\frac{n(n-1)}{2}\right) + c_7\left(\frac{n(n-1)}{2}\right) + c_8(n-1) \\
 &= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2}\right)n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8\right)n \\
 &\quad - (c_2 + c_4 + c_5 + c_8) .
 \end{aligned}$$

$$T(n) = a \cdot n^2 + b \cdot n + c$$

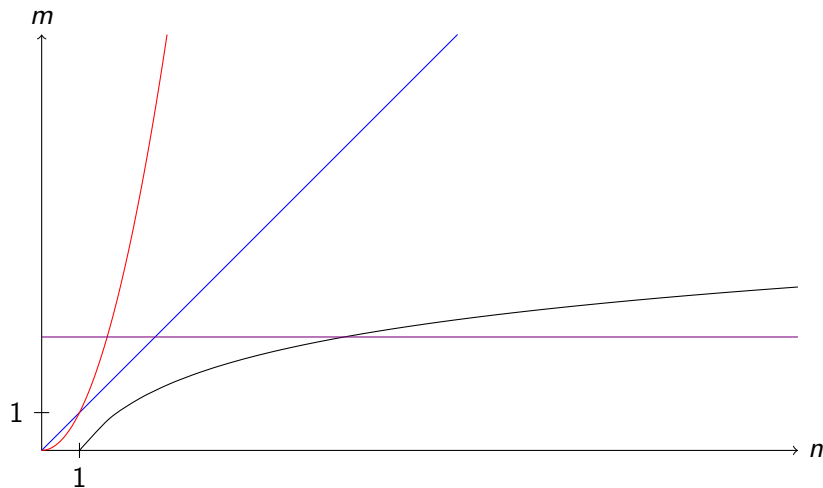
# polynomial vs. linear vs. logarithmic vs. constant time



$n$  – input size

$m$  – running time

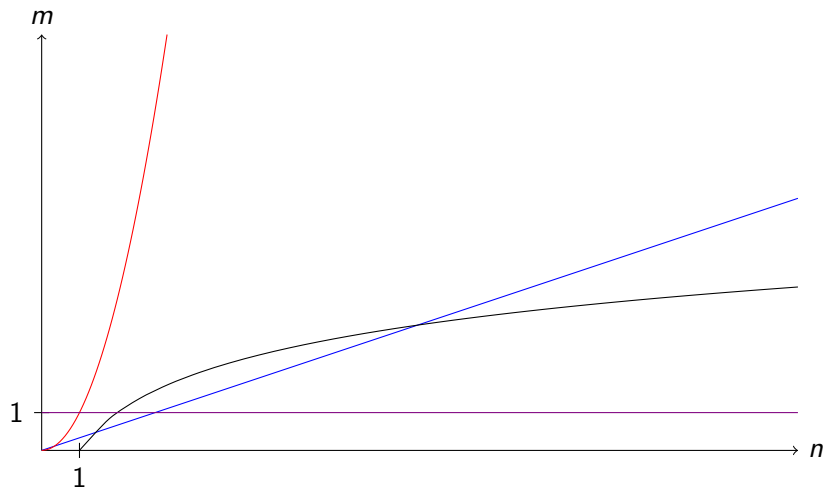
# Change constant time



$n$  – input size

$m$  – running time

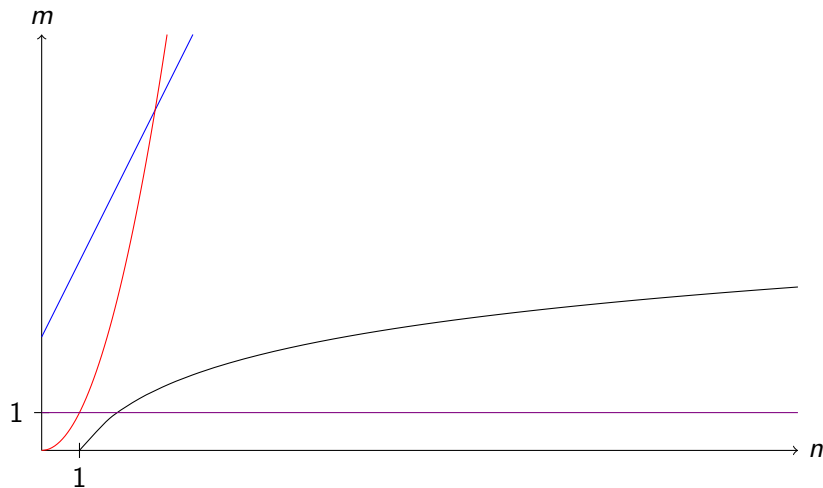
# Change linear time – reduce



$n$  – input size

$m$  – running time

# Change linear time – increase



$n$  – input size

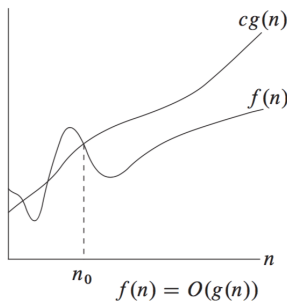
$m$  – running time

# Asymptotic upper bound – the $\mathcal{O}$ -notation

## Definition

For a given function  $g(n)$ , we denote by  $\mathcal{O}(g(n))$  the set of functions

$\mathcal{O}(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq c * g(n) \text{ for all } n \geq n_0\}.$





# Asymptotic upper bound – examples

## Definition

For a given function  $g(n)$ , we denote by  $\mathcal{O}(g(n))$  the set of functions

$\mathcal{O}(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq c * g(n) \text{ for all } n \geq n_0\}.$

## Example

$3*n+8=\mathcal{O}(n).$

Let  $c=4$  and  $n_0=8$ .  $4*n \geq 3*n+8$  if  $n \geq 8$ .

## Example

$2*n^2+4*n=\mathcal{O}(n^2).$

Let  $c=3$  and  $n_0=4$ .  $3*n^2 \geq 2*n^2+4*n$  if  $n \geq 4$ .

Alternative solution: Let  $c=4$  and  $n_0=2$ .  $4*n^2 \geq 2*n^2+4*n$  if  $n \geq 2$ .

# Asymptotic upper bound – examples

## Example

$$2*n^2 + 4*n = O(n^2).$$

Let  $c=3$  and  $n_0=4$ .  $3*n^2 \geq 2*n^2 + 4*n$  if  $n \geq 4$ .

Alternative solution: Let  $c=4$  and  $n_0=2$ .  $4*n^2 \geq 2*n^2 + 4*n$  if  $n \geq 2$ .

|               | 1 | 2  | 3  | 4  | 5  |
|---------------|---|----|----|----|----|
| $2*n^2 + 4*n$ | 6 | 16 | 30 | 48 | 70 |
| $3*n^2$       | 3 | 12 | 27 | 48 | 75 |

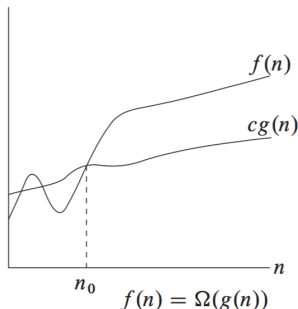
|               | 1 | 2  | 3  | 4  | 5   |
|---------------|---|----|----|----|-----|
| $2*n^2 + 4*n$ | 6 | 16 | 30 | 48 | 70  |
| $4*n^2$       | 4 | 16 | 36 | 64 | 100 |

# Asymptotic lower bound – the $\Omega$ -notation

## Definition

For a given function  $g(n)$ , we denote by  $\Omega(g(n))$  the set of functions

$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq c * g(n) \leq f(n) \text{ for all } n \geq n_0\}.$



# Asymptotic lower bound – examples

## Definition

For a given function  $g(n)$ , we denote by  $\Omega(g(n))$  the set of functions

$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq c * g(n) \leq f(n) \text{ for all } n \geq n_0\}.$

## Example

$2 * n - 6 = \Omega(n).$

Let  $c=1$  and  $n_0=6$ .  $n \leq 2 * n - 6$  if  $n \geq 6$ .

## Example

$n^2 - 3 * n = \Omega(n^2).$

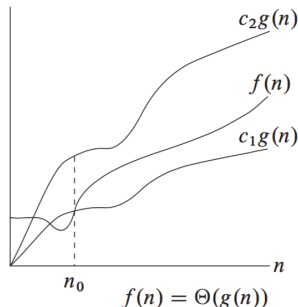
Let  $c=0.5$  and  $n_0=6$ .  $\frac{n^2}{2} \leq n^2 - 3 * n$  if  $n \geq 6$ .

# The $\Theta$ -notation

## Definition

For a given function  $g(n)$ , we denote by  $\Theta(g(n))$  the set of functions

$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2 \text{ and } n_0 \text{ such that } 0 \leq c_1 * g(n) \leq f(n) \leq c_2 * g(n) \text{ for all } n \geq n_0\}$ .



# The $\Theta$ -notation – example

## Definition

For a given function  $g(n)$ , we denote by  $\Theta(g(n))$  the set of functions

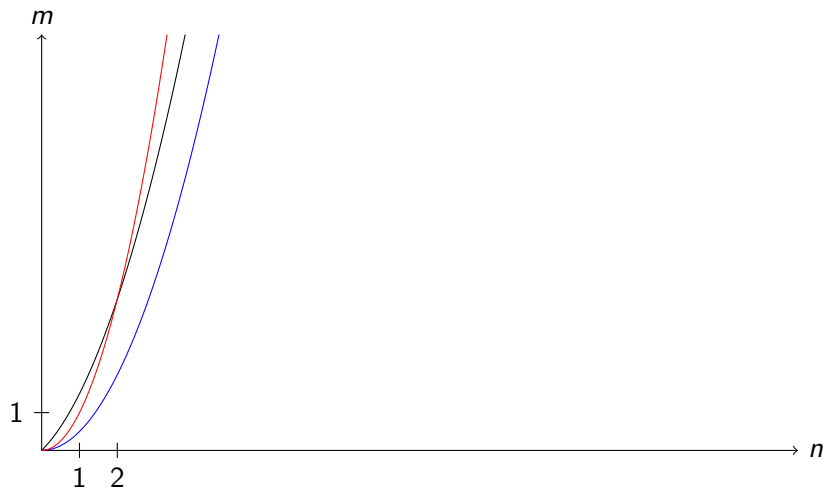
$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2 \text{ and } n_0 \text{ such that } 0 \leq c_1 * g(n) \leq f(n) \leq c_2 * g(n) \text{ for all } n \geq n_0\}.$

## Example

$$\frac{n^2}{2} + n = \Theta(n^2).$$

Let  $c_1 = 0.5$ ,  $c_2 = 1$  and  $n_0 = 2$ .  $\frac{n^2}{2} \leq \frac{n^2}{2} + n \leq n^2$  if  $n \geq 2$ .

# The $\Theta$ -notation – example



$n$  – input size

$m$  – running time

# The $\Theta$ -notation

## Definition

For a given function  $g(n)$ , we denote by  $\Theta(g(n))$  the set of functions

$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2 \text{ and } n_0 \text{ such that } 0 \leq c_1 * g(n) \leq f(n) \leq c_2 * g(n) \text{ for all } n \geq n_0\}.$

## Theorem

For any two functions  $f(n)$  and  $g(n)$ , we have  $f(n) = \Theta(g(n))$  if and only if  $f(n) = \mathcal{O}(g(n))$  and  $f(n) = \Omega(g(n))$ .

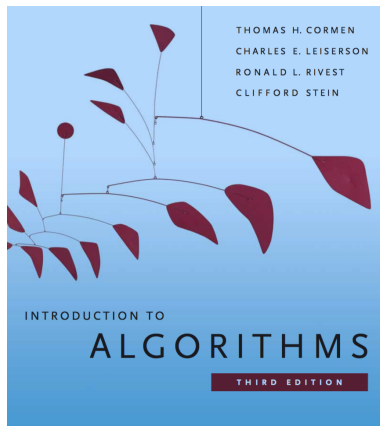


# Computation power – comparison of running times

We assume a 30 day month and 365 day year.

|            | 1 Second            | 1 Minute             | 1 Hour                | 1 Day                     | 1 Month                    | 1 Year                      | 1 Century                    |
|------------|---------------------|----------------------|-----------------------|---------------------------|----------------------------|-----------------------------|------------------------------|
| $\lg n$    | $2^{1 \times 10^6}$ | $2^{6 \times 10^7}$  | $2^{3.6 \times 10^9}$ | $2^{8.64 \times 10^{10}}$ | $2^{2.592 \times 10^{12}}$ | $2^{3.1536 \times 10^{13}}$ | $2^{3.15576 \times 10^{15}}$ |
| $\sqrt{n}$ | $1 \times 10^{12}$  | $3.6 \times 10^{15}$ | $1.29 \times 10^{19}$ | $7.46 \times 10^{21}$     | $6.72 \times 10^{24}$      | $9.95 \times 10^{26}$       | $9.96 \times 10^{30}$        |
| $n$        | $1 \times 10^6$     | $6 \times 10^7$      | $3.6 \times 10^9$     | $8.64 \times 10^{10}$     | $2.59 \times 10^{12}$      | $3.15 \times 10^{13}$       | $3.16 \times 10^{15}$        |
| $n \lg n$  | 62746               | 2801417              | 133378058             | 2755147513                | 71870856404                | 797633893349                | $6.86 \times 10^{13}$        |
| $n^2$      | 1000                | 7745                 | 60000                 | 293938                    | 1609968                    | 5615692                     | 56176151                     |
| $n^3$      | 100                 | 391                  | 1532                  | 4420                      | 13736                      | 31593                       | 146679                       |
| $2^n$      | 19                  | 25                   | 31                    | 36                        | 41                         | 44                          | 51                           |
| $n!$       | 9                   | 11                   | 12                    | 13                        | 15                         | 16                          | 17                           |

# References



This work was supported by the construction EFOP-3.4.3-16-2016-00021. The project was supported by the European Union, co-financed by the European Social Fund.